Complete Log4j2 Configuration

Java + Selenium + TestNG + Maven framework

1. Project structure

```
java-selenium-testng-comprehensive/
■■■ pom.xml
■■■ src/
    ■■■ main/
    ■   ■■■ java/       (framework classes)
    ■   ■■■ resources/
    ■       ■■■ log4j2.xml
    ■■■ test/
        ■■■ java/       (TestNG tests and listeners)
```

2. Maven dependencies

Add the Log4j2 API and Core dependencies to pom.xml. Keep both modules on the same compatible, patched version. The version below is an example; align it with your project dependency policy.

```
<properties>
    <log4j2.version>2.25.2</log4j2.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>${log4j2.version}</version>
    </dependency>

    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>${log4j2.version}</version>
    </dependency>
</dependencies>
```

3. Complete log4j2.xml

Create src/main/resources/log4j2.xml. This configuration logs to the console and a rolling file, rotates by size or day, and reduces noise from common third-party libraries.

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" monitorInterval="60">

    <Properties>
        <Property name="LOG_DIR">target/logs</Property>
        <Property name="LOG_PATTERN">%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level [%t] %c{1} - %msg%n</Property>
    </Properties>

    <Appenders>
        <Console name="ConsoleAppender" target="SYSTEM_OUT">
            <PatternLayout pattern="${LOG_PATTERN}"/>
        </Console>

        <RollingFile name="FileAppender"
                     fileName="${LOG_DIR}/automation.log"
                     filePattern="${LOG_DIR}/archive/automation-%d{yyyy-MM-dd}-%i.log.gz">
            <PatternLayout pattern="${LOG_PATTERN}"/>
            <Policies>
                <SizeBasedTriggeringPolicy size="10 MB"/>
                <TimeBasedTriggeringPolicy interval="1" modulate="true"/>
            </Policies>
            <DefaultRolloverStrategy max="10"/>
        </RollingFile>
    </Appenders>

    <Loggers>
        <Logger name="org.openqa.selenium" level="WARN"/>
        <Logger name="org.apache.http" level="WARN"/>
        <Logger name="io.restassured" level="INFO"/>
```

```
        <!-- Replace com.framework with your actual base package -->
        <Logger name="com.framework" level="DEBUG" additivity="false">
            <AppenderRef ref="ConsoleAppender"/>
            <AppenderRef ref="FileAppender"/>
        </Logger>

        <Root level="INFO">
            <AppenderRef ref="ConsoleAppender"/>
            <AppenderRef ref="FileAppender"/>
        </Root>
    </Loggers>
</Configuration>
```

Important: replace com.framework with your framework's actual Java package. Otherwise, the package-specific DEBUG rule will not match your classes.

4. Add a logger to a Java class

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class BaseTest {

    private static final Logger logger =
            LogManager.getLogger(BaseTest.class);

    public void setUp() {
        logger.info("Test setup started");
        // Initialize browser and test configuration
        logger.info("Test setup completed");
    }

    public void tearDown() {
        logger.info("Test teardown started");
        // Close browser and clean up resources
        logger.info("Test teardown completed");
    }
}
```

Create a static final logger in each class that needs logging. Log meaningful events rather than every line of code. Never log passwords, tokens, cookies, or other secrets.

5. TestNG listener logging

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class TestListener implements ITestListener {

    private static final Logger logger =
            LogManager.getLogger(TestListener.class);

    @Override
    public void onTestStart(ITestResult result) {
        logger.info("TEST STARTED: {}", result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        logger.info("TEST PASSED: {}", result.getName());
    }

    @Override
    public void onTestFailure(ITestResult result) {
        logger.error("TEST FAILED: {}", result.getName());
        if (result.getThrowable() != null) {
            logger.error("Failure details:", result.getThrowable());
        }
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        logger.warn("TEST SKIPPED: {}", result.getName());
    }
}
```

Register the listener using your existing testng.xml configuration or @Listeners annotation. Avoid registering it twice, which can produce duplicate lifecycle logs.

6. Logging levels

Level	Purpose	Example
TRACE	Very detailed diagnostics	logger.trace("Detailed diagnostic information");
DEBUG	Technical troubleshooting detail	logger.debug("Debugging information");
INFO	Normal execution milestones	logger.info("Test execution started");
WARN	Unexpected condition; execution may continue	logger.warn("Unexpected condition encountered");
ERROR	Failure needing investigation	logger.error("Test execution failed");
FATAL	Severe application failure	logger.fatal("Critical failure");

With the root level set to INFO, INFO, WARN, ERROR, and FATAL messages are normally emitted. DEBUG and TRACE are filtered out unless a more specific logger is configured at a lower threshold.

7. Run and verify

```
mvn clean test
```

Look for the log file at target/logs/automation.log. Archived logs are written under target/logs/archive/ when rollover happens.

8. Troubleshooting checklist

• No file created: verify log4j2.xml is under src/main/resources and included on the runtime classpath.
• Only console output: ensure FileAppender is referenced by the relevant logger or root logger.
• DEBUG messages missing: check the effective logger level and package name.
• Duplicate entries: check logger additivity and avoid duplicate listener registration.
• Listener messages missing: ensure TestNG is actually registering the listener.
• Sensitive values: do not log credentials, tokens, cookies, or personal data.

Design note: Log4j2 is for diagnostic execution logs. Allure is a separate reporting tool for test results, steps, and attachments.